

Nome e cognome _____ Matricola _____

La durata della prova è di 2 ore. Scrivere in modo leggibile. E' obbligatorio restituire il testo e tutti i fogli forniti, anche se non si consegna il compito.

1. Considerati CPU-burst time (in ms) del set di processi descritto in tabella

Processo	Burst Time	Tempo di arrivo
P1	6	0
P2	5	1
P3	3	3
P4	4	4
P5	2	5

- a) Disegnare 2 diagrammi di Gantt che illustrino l'esecuzione di questi processi usando gli algoritmi di Shortest Remaining Time First con prelazione (SRTF) e RR (quanto = 3 ms). Indicare l'istante di tempo di ciascuna commutazione fra un processo e l'altro
- b) Calcolare il tempo di attesa per ciascun processo e per ciascun algoritmo di scheduling.
- c) Calcolare il tempo di risposta per ciascun processo e per ciascun algoritmo di scheduling
- d) Discutere vantaggi e svantaggi dei due algoritmi

2. Tenuto conto che:

- il pid del processo padre è **9** e quello del primo figlio è **10**, quello del secondo figlio è **11**, quello del figlio del primo figlio è **12**.
- **Cosa stamperà il seguente programma? Giustificare in estrema sintesi la risposta**

```
int var1 = 3;
```

```
int main(){
    int var2 = 5;
    pid_t pid1, pid2;

    printf("Sono il processo padre.\n");

    pid1 = fork();
    var = var + 2;

    if(pid1 == 0) {
        pid2 = fork();
        var1 = var2 + pid2;
    }
    else {
        pid2 = fork();
        var2 = var1 + pid1 + pid2;
    }
    printf("pid = %d var1 = %d, var2 = %d \n", getpid(), var1, var2);
    return 0;
}
```

3. Dato il frammento di programma che segue:

- a) Discutere le sincronizzazioni presenti in questo programma
- b) Cosa stamperà il programma e perché?
- c) Cos'è una variabile di condizione e che ruolo ha nel programma?

```
#define NUM_THREADS 2
```

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

```

pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int var = 1, turn = 0;

void *func1(void* param){
    pthread_mutex_lock(&mutex);
    while (turn == 0) {pthread_cond_wait(&cond, &mutex);}
    var += turn;
    printf("func1: var = %d, turn = %d\n", var, turn);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

void *func2(void* param){
    pthread_mutex_lock(&mutex);
    turn = var;
    printf("func2: var = %d, turn = %d\n", var, turn);
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

int main(){
    pthread_t threads[NUM_THREADS];

    pthread_create(&threads[0], NULL, func1, NULL);
    pthread_create(&threads[1], NULL, func2, NULL);
    pthread_join(threads[0], NULL);
    pthread_join(threads[1], NULL);

    printf("Final values: var = %d, turn = %d\n", var, turn);
    return 0;
}

```

4. Domande a risposta multipla:

Selezionare le risposte corrette e giustificarle

- a. **Quale delle seguenti affermazioni descrive correttamente l'architettura a microkernel in un sistema operativo?**
 - 1) Tutte le funzionalità del SO sono implementate nel kernel principale.
 - 2) Il microkernel fornisce solo le funzionalità di base, come la gestione dei thread e la comunicazione inter-processo, delegando altre funzionalità al livello utente.
 - 3) Un SO a microkernel è più efficiente di un monolitico poiché tutte le funzioni sono centralizzate nel kernel.
 - 4) I microkernel sono meno sicuri rispetto ai monolitici, poiché il codice kernel è eseguito in modo utente.
- b. **Quale delle seguenti affermazioni è vera riguardo all'algoritmo di schedulazione Rate-Monotonic (RM)?**
 - 1) Assegna priorità più alte ai task con deadline più ravvicinate.
 - 2) Assegna priorità più alte ai task che si ripetono più frequentemente.
 - 3) È un algoritmo di schedulazione a priorità dinamica.
 - 4) È adatto per sistemi real-time.
- c. **Quale delle seguenti affermazioni è corretta riguardo l'architettura NUMA?**
 - 1) La memoria è condivisa uniformemente tra tutti i processori, con lo stesso tempo di accesso per ogni CPU.
 - 2) In un sistema NUMA, ogni processore ha accesso più veloce alla propria memoria locale rispetto alla memoria di altri processori.
 - 3) NUMA è un modello in cui ogni processore ha una memoria fisica separata e non condivisa.
 - 4) I sistemi NUMA non consentono la parallelizzazione, poiché ogni processore accede alla memoria in modo sequenziale.
- d. **Quale tra i seguenti è un vantaggio dell'uso dei semafori binari rispetto ai mutex per la sincronizzazione tra thread?**
 - 1) I semafori binari sono più veloci da acquisire rispetto ai mutex.
 - 2) I semafori binari possono essere utilizzati per la sincronizzazione tra processi su sistemi multi-core.
 - 3) I semafori binari supportano l'attesa e il rilascio condizionale di risorse condivise, mentre i mutex no.
 - 4) I semafori binari possono essere utilizzati in modo da evitare deadlock, mentre i mutex no.

5. Illustrare schematicamente come viene gestito una chiamata di sistema nelle sue fasi principali.

6. Discutere la differenza tra user thread e kernel thread discutendo i vantaggi e svantaggi dei diversi modelli di mapping.